

Roteiro da call

Desafio AI Engineer — Bill · ~15 min de apresentação + código junto

bill-poc.vercel.app · github.com/PauloRbtoCodes/bill-poc

Antes de começar

5 min, sozinho

- ☐ Abrir **bill-poc.vercel.app** e confirmar que carrega
- ☐ Abrir o repositório no editor
- ☐ Terminal aberto em `~/Documentos/case-bill/apps/api`
- ☐ Mandar o link no chat da call logo no início

Se quiser rodar local também: `docker compose up -d`, depois `uv run uvicorn billpoc.api:app --port 8000` num terminal e `npm run dev` na raiz em outro.

Não rode `npm run build` com o dev server de pé — os dois compartilham `.next` e o dev quebra. Se acontecer: `rm -rf .next` e reinicie.

1. A tese, antes de qualquer código

2 min

DIGA

"Antes de mostrar rodando, a ideia central em uma frase: **num boleto brasileiro, valor e vencimento não precisam ser confiados ao LLM — eles estão aritmeticamente codificados na linha digitável**. Então dá para *conferir* em vez de *confiar*."

Mostre no terminal:

```
uv run python -c "  
from billpoc.validate.boleto import decodificar  
b = decodificar('34191790010104351004791020150008115700000123456')  
print('valor      ', b.valor)  
print('vencimento', b.vencimento)  
print('válido     ', b.valido)  
"
```

DIGA

"Isso é aritmética pura, sem modelo nenhum. O LLM continua fazendo o trabalho difícil — ler PDF de layout desconhecido, decidir se é cobrança ou ruído, categorizar. Mas nos dois campos onde errar custa dinheiro, existe um piso."

Se perguntarem "e quando não tem código de barras?" → a resposta está no passo 4.

2. Rodando sobre a caixa real

3 min

```
uv run billpoc run --fonte eml
```

Aponte o resumo: **48 e-mails · 42 ruído · 6 cobranças · 0 erros · US\$ 0,36.**

DIGA

"87% da caixa é ruído. Por isso a triagem roda num modelo barato sobre cabeçalho e corpo, sem abrir PDF — e só o que passa vai para o modelo caro com o documento inteiro. É o que faz o custo por e-mail ficar de pé em escala."

Depois, a acurácia:

```
uv run billpoc golden
```

DIGA

"16 casos rotulados à mão. Os pisos são assimétricos de propósito: recall exige 100%, precisão aceita 90%. Um falso positivo o revisor descarta em dois minutos; um falso negativo some, a conta não é paga, e o cliente descobre pela multa."

3. O que a caixa real ensinou

2 min

Este é o trecho que separa de uma POC com dados fabricados.

DIGA

"Três coisas que só apareceram porque rodei nos e-mails de verdade:"

1. **Todas as cobranças chegam encaminhadas** da mesma pessoa. O `From` é do encaminhador. Sem tratar isso, os seis boletos casariam com um "fornecedor" de domínio gmail.com.
2. **Dois dos seis PDFs são protegidos por senha**. Num deles a linha digitável estava no corpo, e a varredura determinística resolveu sem abrir o PDF.
3. **O mesmo provedor aparece com dois nomes** (Starfiber e Lord Fibra) — é por isso que o casamento de fornecedor é por CNPJ, não por nome.

4. A tela — e o momento que importa

4 min

Abra **bill-poc.vercel.app**.

Caixa de entrada

Três faixas. Mostre a aba **Ruído** e a justificativa de cada descarte.

DIGA

"O ruído fica visível de propósito, com o motivo e um botão para contestar. Descartar em silêncio é criar um falso negativo que ninguém descobre."

Revisão — abra a Starfiber ou a Lord Fibra

Aponte os selos de origem ao lado de cada campo.

DIGA

"Esta é a informação mais importante da tela. 🔒 quer dizer que o número foi conferido por aritmética; 🤖 que foi lido pelo modelo. Sem essa distinção o revisor confere tudo com o mesmo cuidado, que é o mesmo que não conferir nada."

Repare que **valor** está 🔒 e **vencimento** está 🤖:

DIGA

"O formato codifica o valor mas não o vencimento — e o sistema *sabe* que não pode conferir a data. Por isso caiu em revisão mesmo com o valor confirmado. Essa é a resposta para 'e quando não tem como conferir': não quebra, escala de confiança."

O conflito — use a caixa de demonstração

```
uv run billpoc semear
uv run billpoc run --fonte demo --somente-cache
```

Abra a **Norte Logística**: o modelo lê R\$ 987,00, o código de barras diz R\$ 9.870,00. Os dois aparecem lado a lado, com o determinístico já gravado.

O aprendizado — Limpeza Total

Quatro mensalidades do mesmo fornecedor. Abra a de setembro e mostre **Recorrência**.

DIGA

"As três primeiras saíram como 'único' com selo 🤖 — o modelo lê um documento isolado e não tem como saber. A quarta virou 'recorrente' com selo 📊 e a evidência 'três cobranças anteriores em cadência mensal'. O sistema fica melhor com o uso, e não por treinar nada: o histórico é determinístico."

Agenda

Agrupamento por vencimento e o botão **Exportar** → **CNAB 240**.

DIGA

"Copiar-e-colar resolve três contas. Trinta é o volume de um cliente pequeno de verdade, e aí o caminho é subir um arquivo de remessa no banco."

5. Mexer no código junto

o resto do tempo

Três lugares bons, do mais simples ao mais interessante:

a) Um limiar da política

`apps/api/src/billpoc/validate/rules.py`, no topo:

```
LIMIAR_TRIAGEM = 0.85  
LIMIAR_CAMPO = 0.70
```

Baixar `LIMIAR_CAMPO` para 0.4 e rodar de novo mostra contas saindo da revisão. Boa conversa sobre onde calibrar.

b) O prompt

`apps/api/src/billpoc/extract/prompts.py`. Mexer nele muda o `PROMPT_VERSION`, o cache é invalidado, e `billpoc golden` mede se melhorou ou piorou. É a demonstração do ciclo de iteração.

c) A tolerância do alerta de valor

`apps/api/src/billpoc/enrich.py`:

```
MINIMO_PARA_RECORRENCIA = 3  
TOLERANCIA_PADRAO = Decimal("15.0")
```

Baixar para 2% faz aparecer alerta em variação normal de conta de consumo — boa conversa sobre o limite entre sinal e ruído.

Perguntas prováveis

PERGUNTA	RESPOSTA CURTA
"Por que não usou IMAP?"	Testei; o Google recusa senha de conta desde 2022. OAuth é também o caminho de produção.
"Isso escala para 100 clientes?"	OAuth por caixa não escala. Um endereço de encaminhamento por cliente elimina o consentimento e funciona em qualquer provedor. MailSource já é a interface.
"Quanto custa?"	US\$ 0,36 para 48 e-mails. billpoc report quebra por etapa e modelo.
"E se o modelo alucinar?"	Nos campos com código de barras, é detectado. Nos outros, cai em revisão por falta de corroboração. Nada paga sozinho.
"Por que não um agente?"	Extrair campos é uma chamada com schema estrito, não um loop. Agente aqui adiciona não-determinismo sem adicionar capacidade.
"Como sabe que uma mudança melhorou?"	billpoc golden . Sem isso é opinião.
"E prompt injection?"	O prompt trata documento como dado, e a política de decisão está em código, não no prompt. Um PDF não consegue se auto-aprovar.
"Por que Postgres e não planilha?"	A auditoria é relacional: um payable tem N campos extraídos, N verificações e N ações de revisão. Em abas isso vira cópia manual.

Se algo der errado ao vivo

SINTOMA	O QUE FAZER
Site fora do ar	Rodar local: a demo inteira funciona em localhost
API não responde	uv run billpoc doctor mostra o que caiu
Sem chave / sem rede	uv run billpoc run --fonte demo --somente-cache
Banco sujo	truncate email_messages, vendors cascade; e rodar de novo
Resetar tudo	docker compose down -v && docker compose up -d && uv run billpoc initdb

Contexto sobre a empresa

O site é **oibill.ai** (oibill.com.br redireciona). O produto público é um assistente no WhatsApp para **contas a receber** — cobrança recorrente de clientes.

DIGA

"Vi que o produto de vocês hoje é o lado do *receber*; isto aqui é o *pagar*."

E um detalhe que apareceu nos próprios dados: os e-mails de "Conciliação Maio/2026" na caixa de teste vieram de **pedro@oibill.com.br** — o pipeline classificou todos como ruído, corretamente. Vale citar que `payment_schedules` guarda banco, data e **protocolo** justamente para reconciliar depois.

Se sobrar tempo — o que eu faria a seguir

Aprender template por fornecedor a partir das correções humanas. `review_actions` é o sinal de treino: depois de N extrações confirmadas do mesmo remetente e layout, promover para parser determinístico e tirar aquele fornecedor do caminho do LLM.

DIGA

"O custo cai e a confiança sobe com o uso — que é a curva certa para um produto que ganha clientes."